PROYECTO INTEGRADOR

# PrestaLab

*Sistema de reserva y control de préstamo de equipo y espacios de laboratorio*

## ENTREGA 4 — Diseño de la información y modelo de datos

| | |
|---|---|
| **Carrera** | Ingeniería en Software |
| **Materia** | Arquitectura Orientada a Servicios |
| **Docente** | Ing. Luis Leonardo Mendoza Reséndiz |
| **Equipo** | Tranquilo nos la navegamos |
| **Integrantes** | Jhonatan Uriel Morales Badillo, José Vidal López Salazar, David Sosa Gómez, Alejandro Ramírez Contreras |
| **Periodo / Semana** | Septiembre – Diciembre 2026 · Semana 4 de 8 |
| **Repositorio** | https://github.com/JhonaRaida/prestalab · tag: entrega-4 |

# Reporte breve de la entrega

| | |
|---|---|
| **Entrega** | 4 — Diseño de la información y modelo de datos |
| **Nombre del proyecto** | PrestaLab |
| **Integrantes** | Jhonatan Uriel Morales Badillo, José Vidal López Salazar, David Sosa Gómez, Alejandro Ramírez Contreras |
| **Objetivo de la semana** | Definir cómo se guarda la información del sistema (tablas, llaves, relaciones y reglas) y dejar scripts que crean la base con datos de prueba. |
| **Qué se diseñó** | 10 tablas en 4 bases de datos (una por servicio), diagrama entidad-relación, diccionario de datos, scripts SQL, 54 registros de prueba y consultas de verificación. |
| **Qué cambió respecto a la entrega anterior** | La API de la Entrega 3 no cambió. Se agregó la tabla roles (antes los roles eran solo una lista) y los datos de prueba pasaron a archivos SQL. |
| **Problemas encontrados** | (1) No se puede poner una llave foránea entre tablas de bases distintas. (2) Hay que evitar que dos personas reserven lo mismo a la misma hora. (3) Los datos de prueba con fechas fijas se vuelven viejos. |
| **Cómo se resolvieron** | (1) Se guarda solo el id del otro servicio y la API comprueba que exista. (2) Una restricción de PostgreSQL rechaza horarios encimados (sección 4). (3) Las fechas de prueba se calculan a partir de «hoy». |
| **Pendientes para la siguiente semana** | Entrega 5: evidencias de CRUD, colección de Postman y README. |
| **Repositorio y versión** | https://github.com/JhonaRaida/prestalab · tag: entrega-4 |

# 1. ¿Qué se guarda y dónde?

PrestaLab está dividido en servicios. Cada servicio que guarda información tiene **su propia base de datos** y ningún otro servicio la toca: si necesita datos de otro, se los pide por la API. El servicio de Reportes no tiene base; arma sus resultados con datos de Reservas e Inventario.

| Base de datos | Servicio | Tablas | Qué guarda |
|---|---|---|---|
| **usuarios_db** | Usuarios y Autenticación | roles, usuarios, sanciones | Cuentas, roles y bloqueos por retraso |
| **inventario_db** | Inventario | laboratorios, categorias, recursos | Laboratorios y el equipo o espacios que se prestan |
| **reservas_db** | Reservas y Préstamos | reservas, prestamos, incidencias | Apartados, entregas, devoluciones y daños |
| **notificaciones_db** | Notificaciones | notificaciones | Avisos a los usuarios |

### Decisiones principales

- **Una base por servicio:** un servicio puede cambiar sin romper a otro.
- **Ids numéricos automáticos** (SERIAL), igual que en las rutas de la API (`/reservas/12`).
- **Fechas con zona horaria** (TIMESTAMPTZ) para comparar horarios sin errores.
- **Contraseñas cifradas** (hash bcrypt), nunca en texto normal.
- **No se pierde el historial:** un recurso dado de baja solo cambia de estado y nada se borra en cascada.

# 2. Diagrama entidad-relación

La Figura 1 (siguiente página) muestra las 10 tablas agrupadas por base de datos.

### Cómo leer el diagrama

- **PK** = llave primaria · **FK** = llave foránea · **UQ** = valor único.
- **Línea negra:** relación dentro de la misma base; PostgreSQL la verifica.
- **Línea roja punteada:** relación entre bases distintas (marcada como FK*); la verifica la API.

## 2.1 Relaciones

Hay 6 relaciones con llave foránea real y 10 relaciones lógicas entre servicios. Todas son de uno a muchos (1:N), excepto reserva → préstamo (una reserva tiene cero o un préstamo). No hay relaciones muchos a muchos ni ambiguas.

| Relación | Tipo | Significado |
|---|---|---|
| **roles → usuarios** | FK | Cada usuario tiene un rol del catálogo. |

| Relación | Tipo | Significado |
|---|---|---|
| **usuarios → sanciones** | FK | Un usuario puede tener varias sanciones. |
| **categorias → recursos** | FK | Cada recurso pertenece a una categoría. |
| **laboratorios → recursos** | FK | Cada recurso pertenece a un laboratorio. |
| **reservas → prestamos** | FK (1 : 0..1) | Una reserva genera como máximo un préstamo. |
| **prestamos → incidencias** | FK | Un préstamo puede tener varias incidencias. |
| **usuarios → reservas, prestamos, notificaciones** | Lógica | Quién reserva, quién entrega o recibe y a quién se avisa. |
| **recursos, laboratorios → reservas, incidencias** | Lógica | Qué recurso se reservó o tuvo un problema. |
| **usuarios → laboratorios** | Lógica | Encargado de cada laboratorio. |
| **prestamos → sanciones** | Lógica | Préstamo que causó la sanción. |

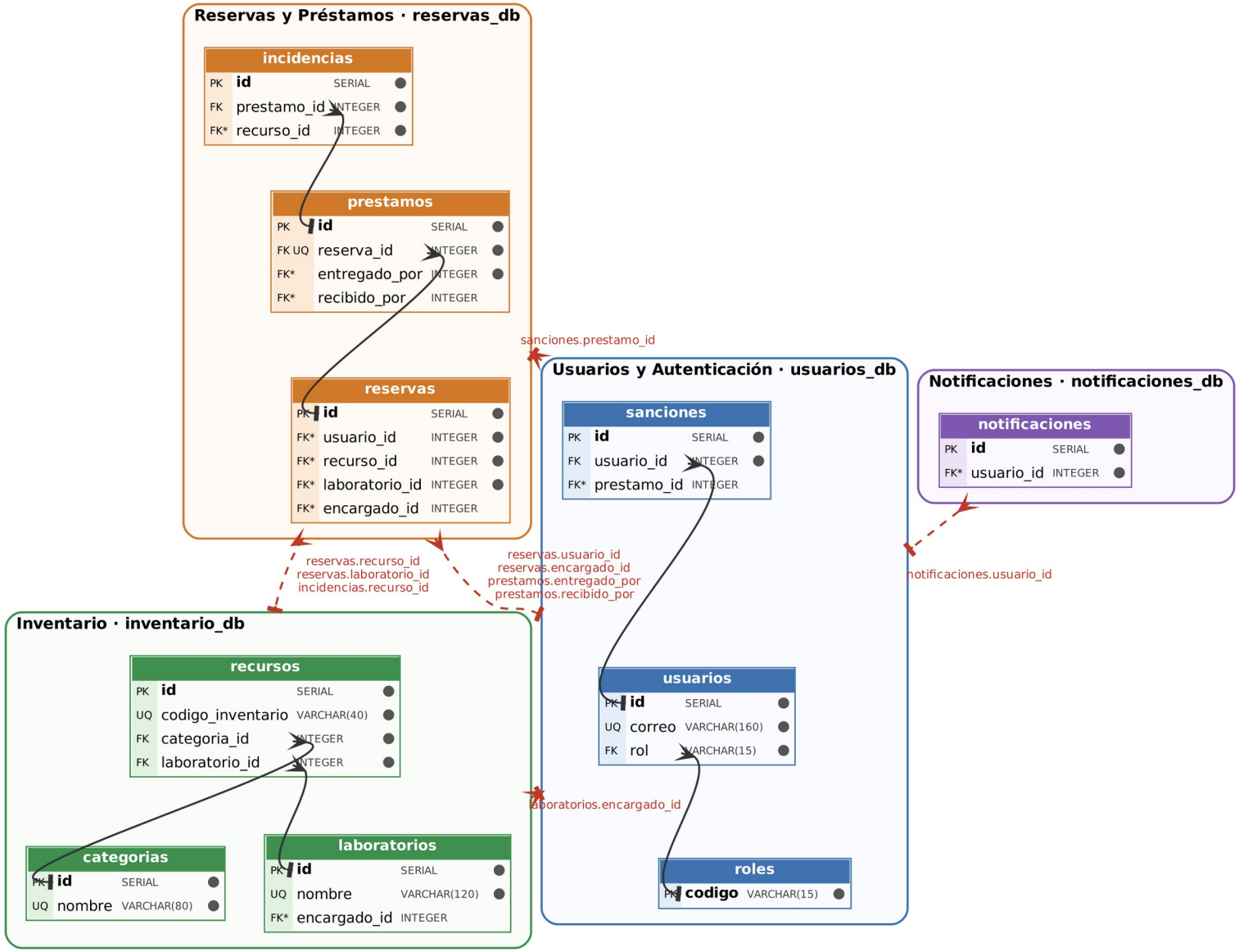



*Figura 1. Diagrama entidad-relación de PrestaLab.*

## 3. Diccionario de datos

Todas las columnas son obligatorias (NOT NULL) salvo las marcadas como **opcional**.

### roles · usuarios_db

Catálogo de los 4 roles.

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **codigo** | VARCHAR(15) | PK | alumno, docente, encargado o administrador |
| **descripcion** | VARCHAR(160) | — | Qué puede hacer el rol |

### usuarios · usuarios_db

Personas con cuenta en el sistema.

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **id** | SERIAL | PK | Identificador |
| **nombre** | VARCHAR(120) | — | Nombre completo |
| **correo** | VARCHAR(160) | UNIQUE | Correo para iniciar sesión |
| **matricula** | VARCHAR(20) | opcional | Matrícula o clave de empleado |
| **contrasena** | VARCHAR(100) | — | Contraseña cifrada (hash bcrypt) |
| **rol** | VARCHAR(15) | FK → roles; default alumno | Rol del usuario |
| **estado** | VARCHAR(10) | CHECK: activo, bloqueado | Si la cuenta puede usarse |
| **creado_en** | TIMESTAMPTZ | default now() | Fecha de registro |

### sanciones · usuarios_db

Bloqueos temporales por devolver tarde.

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **id** | SERIAL | PK | Identificador |
| **usuario_id** | INTEGER | FK → usuarios | Usuario sancionado |
| **motivo** | VARCHAR(255) | — | Por qué se sancionó |
| **inicio** | TIMESTAMPTZ | default now() | Inicio del bloqueo |
| **fin** | TIMESTAMPTZ | CHECK fin > inicio | Fin del bloqueo |
| **prestamo_id** | INTEGER | opcional; FK lógica | Préstamo que causó el retraso |

### laboratorios · inventario_db

Laboratorios físicos.

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **id** | SERIAL | PK | Identificador |
| **nombre** | VARCHAR(120) | UNIQUE | Nombre del laboratorio |
| **ubicacion** | VARCHAR(160) | — | Edificio y planta |
| **encargado_id** | INTEGER | opcional; FK lógica | Usuario encargado |

### categorias · inventario_db

Tipos de recurso (laptops, proyectores, aulas...).

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **id** | SERIAL | PK | Identificador |
| **nombre** | VARCHAR(80) | UNIQUE | Nombre de la categoría |
| **tipo** | VARCHAR(10) | CHECK: equipo, espacio | Si es equipo o espacio |

### recursos · inventario_db

Cada equipo o espacio que se puede reservar.

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **id** | SERIAL | PK | Identificador |
| **nombre** | VARCHAR(120) | — | Nombre del recurso |
| **codigo_inventario** | VARCHAR(40) | UNIQUE | Código físico de inventario |
| **categoria_id** | INTEGER | FK → categorias | Categoría |
| **laboratorio_id** | INTEGER | FK → laboratorios | Laboratorio donde está |
| **descripcion** | VARCHAR(500) | opcional | Detalles |
| **estado** | VARCHAR(15) | CHECK: activo, mantenimiento, baja | Si se puede reservar |
| **creado_en** | TIMESTAMPTZ | default now() | Fecha de alta |

### reservas · reservas_db

Apartado de un recurso en un horario. Es la tabla central.

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **id** | SERIAL | PK | Identificador |
| **usuario_id** | INTEGER | FK lógica | Quién reserva |
| **recurso_id** | INTEGER | FK lógica | Qué se reserva |
| **laboratorio_id** | INTEGER | FK lógica | Laboratorio del recurso (copia) |
| **encargado_id** | INTEGER | opcional; FK lógica | Encargado del laboratorio (copia) |

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **inicio** | TIMESTAMPTZ | CHECK fin > inicio | Inicio del horario |
| **fin** | TIMESTAMPTZ | CHECK fin > inicio | Fin del horario |
| **estado** | VARCHAR(12) | CHECK: confirmada, en_curso, finalizada, cancelada | Etapa de la reserva |
| **creada_en** | TIMESTAMPTZ | default now() | Fecha de creación |
| **recordatorio_enviado** | BOOLEAN | default false | Para no repetir el recordatorio |
| **retraso_avisado** | BOOLEAN | default false | Para no repetir el aviso de retraso |

### prestamos · reservas_db

Entrega y devolución real de lo reservado.

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **id** | SERIAL | PK | Identificador |
| **reserva_id** | INTEGER | FK → reservas; UNIQUE | Reserva que se entregó |
| **entregado_por** | INTEGER | FK lógica | Encargado que entregó |
| **fecha_entrega** | TIMESTAMPTZ | default now() | Momento de la entrega |
| **recibido_por** | INTEGER | opcional; FK lógica | Encargado que recibió |
| **fecha_devolucion** | TIMESTAMPTZ | opcional | Momento de la devolución |
| **estado_devolucion** | VARCHAR(12) | opcional; CHECK: correcto, danado, incompleto | Cómo se devolvió |
| **observaciones** | VARCHAR(500) | opcional | Notas del encargado |

### incidencias · reservas_db

Problemas en un préstamo: daño, faltante o retraso.

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **id** | SERIAL | PK | Identificador |
| **prestamo_id** | INTEGER | FK → prestamos | Préstamo donde ocurrió |
| **recurso_id** | INTEGER | FK lógica | Recurso afectado (copia) |
| **tipo** | VARCHAR(10) | CHECK: dano, faltante, retraso | Tipo de problema |
| **descripcion** | VARCHAR(500) | — | Qué pasó |
| **estado** | VARCHAR(10) | CHECK: abierta, cerrada | Si sigue pendiente |
| **creada_en** | TIMESTAMPTZ | default now() | Fecha de registro |

### notificaciones · notificaciones_db

Avisos para los usuarios.

| Columna | Tipo | Reglas | Descripción |
|---|---|---|---|
| **id** | SERIAL | PK | Identificador |
| **usuario_id** | INTEGER | FK lógica | Destinatario |
| **tipo** | VARCHAR(15) | CHECK: confirmacion, recordatorio, retraso | Tipo de aviso |
| **mensaje** | VARCHAR(500) | — | Texto del aviso |
| **leida** | BOOLEAN | default false | Si el usuario ya la leyó |
| **estado_envio** | VARCHAR(10) | CHECK: pendiente, enviada, fallida | Si ya se envió |
| **creada_en** | TIMESTAMPTZ | default now() | Fecha de creación |

## 4. Restricciones importantes

Las reglas más importantes las cuida la propia base de datos, aunque haya un error en el código. En total hay 10 llaves primarias, 6 llaves foráneas, 5 UNIQUE, 11 CHECK, 55 columnas NOT NULL y 1 restricción de exclusión.

| Regla | Tipo | Tabla |
|---|---|---|
| **No se puede reservar un recurso en un horario que ya está ocupado.** | EXCLUDE (reservas_sin_traslape) | reservas |
| **El fin va después del inicio.** | CHECK | reservas, sanciones |
| **No hay dos cuentas con el mismo correo.** | UNIQUE | usuarios |
| **No se repite el código de inventario.** | UNIQUE | recursos |
| **Una reserva solo se entrega una vez.** | UNIQUE | prestamos |
| **Los estados y tipos solo aceptan valores válidos.** | CHECK | varias |
| **Solo existen los 4 roles del catálogo.** | FK | usuarios |
| **Todo recurso tiene una categoría y un laboratorio que existen.** | FK | recursos |

**¿Cómo se evitan las reservas encimadas?**

Si la Laptop 2 está reservada de 10:00 a 11:00, PostgreSQL rechaza otra reserva de 10:30 a 11:30, aunque las dos peticiones lleguen al mismo tiempo. Una de 11:00 a 12:00 sí se permite, porque empieza justo cuando termina la otra. Las reservas canceladas o finalizadas no cuentan.

Además se crearon 8 índices para que las búsquedas más comunes sean rápidas (por usuario, laboratorio, categoría y notificaciones sin leer).

# 5. Normalización

| Forma | Qué significa | En PrestaLab |
|---|---|---|
| **1FN** | Cada celda guarda un solo valor. | Una reserva guarda un solo recurso; si se necesitan dos, son dos reservas. |
| **2FN** | Todo depende de la llave completa. | Todas las llaves son de un solo campo, así que se cumple. |
| **3FN** | Ningún dato depende de otro dato que no sea la llave. | El nombre del laboratorio solo está en laboratorios; no se repite en recursos. |

**Excepciones a propósito:** reservas guarda una copia de laboratorio_id y encargado_id, e incidencias una de recurso_id. Así no hay que preguntar a otro servicio en cada búsqueda y se conserva el dato histórico.

# 6. Qué servicio usa cada dato

Regla: cada servicio solo lee y escribe sus propias tablas. Si necesita datos de otro, se los pide por la API.

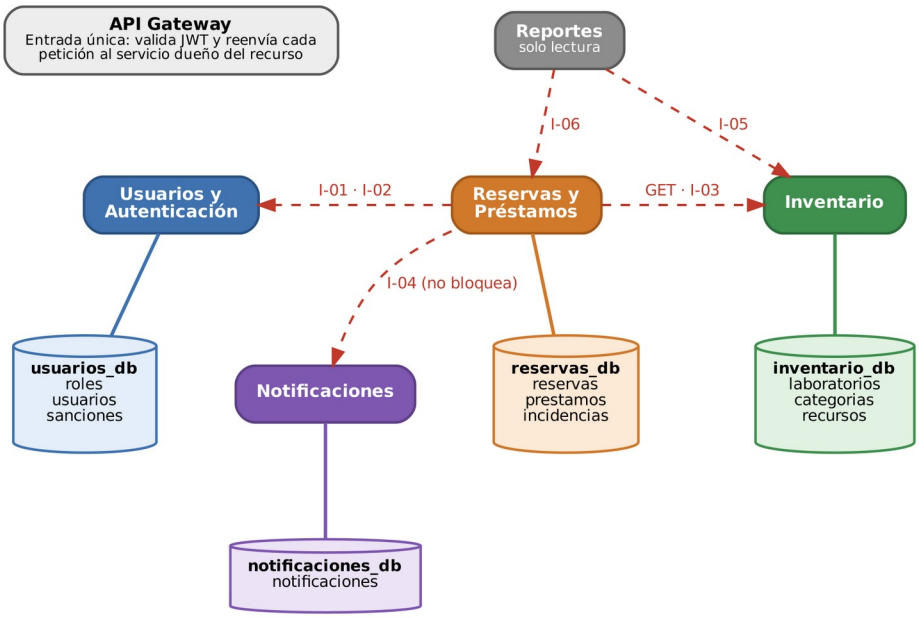



*Figura 2. Datos de cada servicio y consultas entre servicios (líneas rojas punteadas).*

| Servicio | Es dueño de | Pide datos a |
|---|---|---|
| **Usuarios** | roles, usuarios, sanciones | — |
| **Inventario** | laboratorios, categorias, recursos | — |

| Servicio | Es dueño de | Pide datos a |
|---|---|---|
| **Reservas** | reservas, prestamos, incidencias | Usuarios (estado y sanciones), Inventario (recursos) y Notificaciones (avisos) |
| **Notificaciones** | notificaciones | — |
| **Reportes** | ninguna (sin base) | Reservas e Inventario, solo lectura |
| **API Gateway** | ninguna | Solo valida el token y reenvía la petición |

### 6.1 El modelo cubre las operaciones de la API

| Operaciones de la API (Entrega 3) | Tablas que usa |
|---|---|
| **Iniciar sesión, registrarse, gestionar usuarios** | usuarios, roles, sanciones |
| **Laboratorios, categorías y recursos** | laboratorios, categorias, recursos |
| **Disponibilidad y reservas** | reservas (+ recursos y usuarios por API) |
| **Entrega y devolución** | prestamos, reservas, incidencias (+ sanciones por API) |
| **Incidencias** | incidencias |
| **Notificaciones** | notificaciones |
| **Reportes** | reservas, prestamos, incidencias, recursos (por API) |

Los 33 endpoints de la Entrega 3 tienen soporte en el modelo.

## 7. Scripts y datos de prueba

| Archivo (carpeta db/) | Qué hace |
|---|---|
| **<servicio>/001_esquema.sql** | Crea las tablas, llaves, restricciones e índices de cada base. |
| **<servicio>/002_datos_prueba.sql** | Carga los datos de prueba. |
| **instalar.sql** | Crea todo (4 bases, tablas y datos) con un solo comando. |
| **verificar.sql** | Consultas de verificación y pruebas de restricciones. |
| **reiniciar.sql** | Borra las 4 bases para empezar de cero. |

#### Cómo crear la base desde cero

- **Con Docker:** `docker compose up --build` (crea las bases y carga los datos solo).
- **Solo con PostgreSQL:** `psql -U <usuario> -h localhost -d postgres -f db/instalar.sql`

Los scripts se pueden ejecutar varias veces sin error. Las cuentas de prueba usan la contraseña Prestalab123 (solo para pruebas).

### 7.1 Datos de prueba

Hay **54 registros** (el mínimo pedido es 10). No son al azar: cubren los casos que hay que probar:

- Préstamos devueltos a tiempo, con retraso, dañado e incompleto.
- Un alumno con sanción vigente que no puede reservar.
- Reservas futuras y una cancelada, para probar disponibilidad y traslapes.
- Dos encargados con laboratorios distintos.

## 8. Evidencia de verificación

Resultados de ejecutar `db/verificar.sql` (salida completa en docs/evidencia-bd.txt).

### 8.1 Registros por tabla

| Tabla | Base | Registros | Servicio |
|---|---|---|---|
| **roles** | usuarios_db | 4 | Usuarios |
| **usuarios** | usuarios_db | 7 | Usuarios |
| **sanciones** | usuarios_db | 2 | Usuarios |
| **laboratorios** | inventario_db | 3 | Inventario |
| **categorias** | inventario_db | 6 | Inventario |
| **recursos** | inventario_db | 12 | Inventario |
| **reservas** | reservas_db | 7 | Reservas |
| **prestamos** | reservas_db | 4 | Reservas |
| **incidencias** | reservas_db | 4 | Reservas |
| **notificaciones** | notificaciones_db | 5 | Notificaciones |
| **Total** | | **54** | |

### 8.2 Consultas básicas

| Consulta | Resultado |
|---|---|
| **B1. Usuarios por rol** | 1 administrador, 3 alumnos, 1 docente, 2 encargados |
| **B2. Sanciones vigentes** | 1 (alumno3, retraso de 140 minutos) |
| **B3. Recursos con su laboratorio y categoría (une 3 tablas)** | 12 recursos; 1 en mantenimiento (Laptop HP 03) |
| **B4. Préstamos con retraso e incidencias** | 4 préstamos; retrasos de 95 y 140 minutos |
| **B5. Recursos ocupados mañana de 10:00 a 11:00** | Recurso 2 |

| Consulta | Resultado |
|---|---|
| **B6. Notificaciones sin leer** | Usuario 4: 2 · Usuario 7: 1 |

## 8.3 Pruebas de restricciones

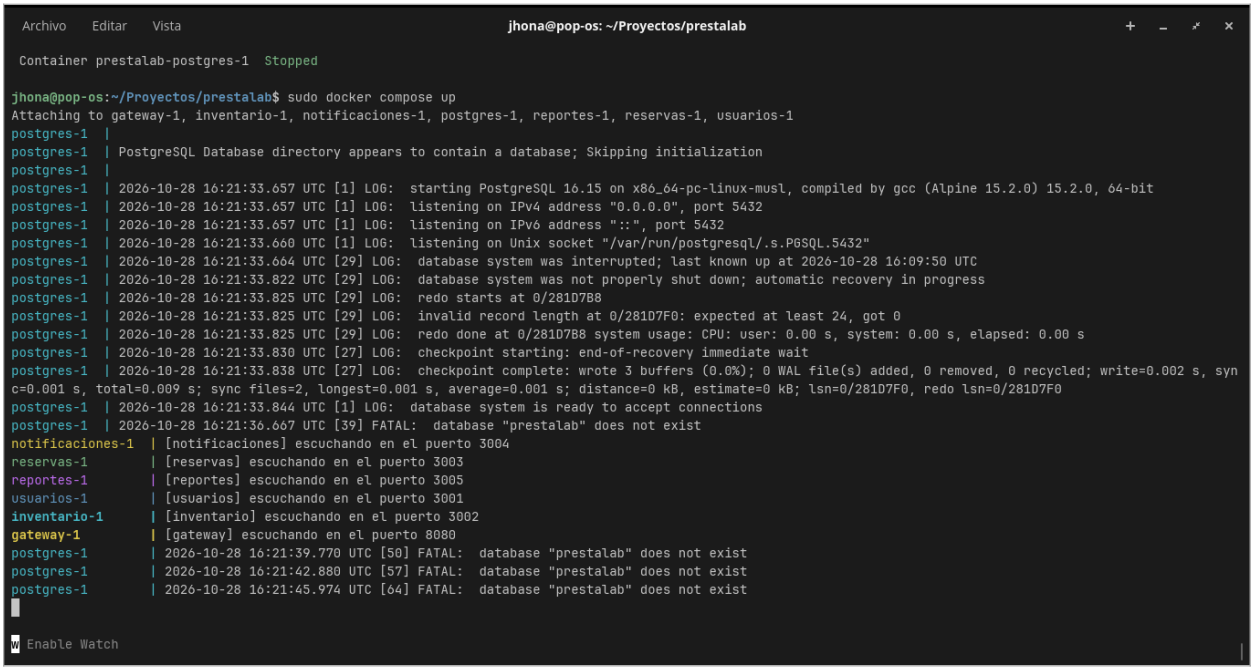
Cada prueba intenta romper una regla. Al final se deshace todo (ROLLBACK), así que no deja datos.

| # | Intento | Resultado |
|---|---|---|
| **C1** | Reservar el recurso 2 en un horario ya ocupado | Rechazado (reservas_sin_traslape) |
| **C2** | Reservar el recurso 2 justo después (11:00–12:00) | Aceptado, como se esperaba |
| **C3** | Préstamo de una reserva que no existe | Rechazado (FK) |
| **C4** | Segundo préstamo para la misma reserva | Rechazado (UNIQUE) |
| **C5** | Reserva con fin antes del inicio | Rechazado (CHECK) |
| **C6** | Tipo de incidencia inválido | Rechazado (CHECK) |
| **C7** | Correo repetido | Rechazado (UNIQUE) |
| **C8** | Rol que no existe | Rechazado (FK) |
| **C9** | Usuario sin nombre | Rechazado (NOT NULL) |
| **C10** | Código de inventario repetido | Rechazado (UNIQUE) |
| **C11** | Estado de recurso inválido | Rechazado (CHECK) |
| **C12** | Recurso con categoría inexistente | Rechazado (FK) |

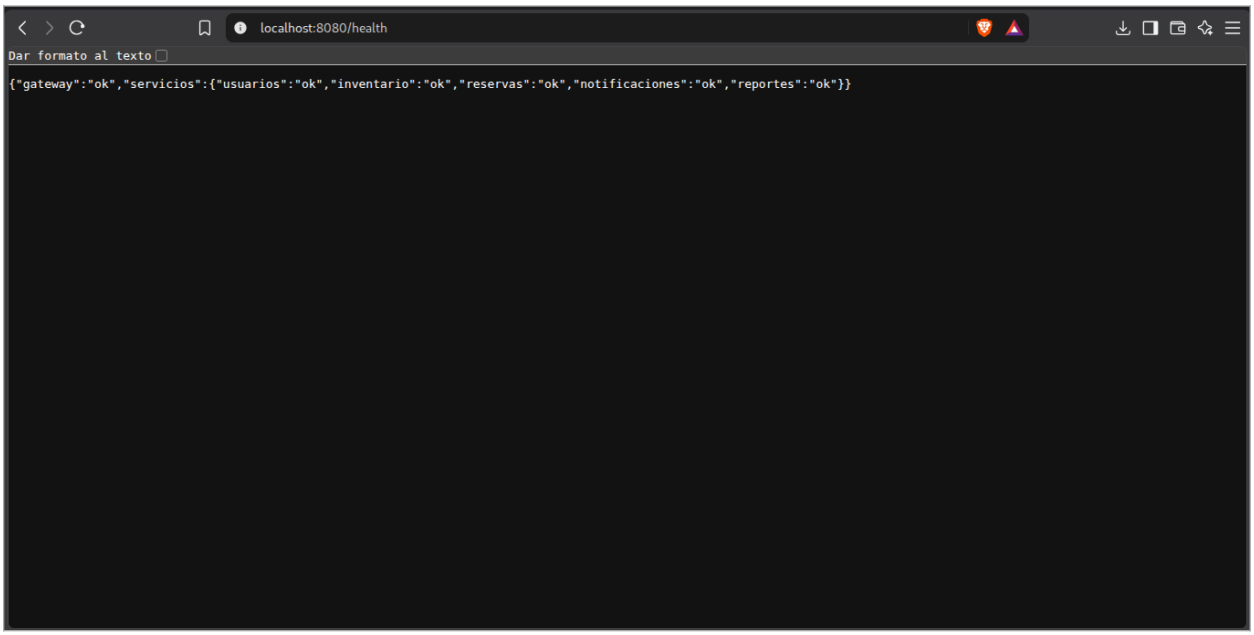
Además, el sistema completo pasó **57 de 57 pruebas automáticas** y las **61 peticiones de Postman** usando estas mismas tablas.

## 8.4 Capturas del equipo

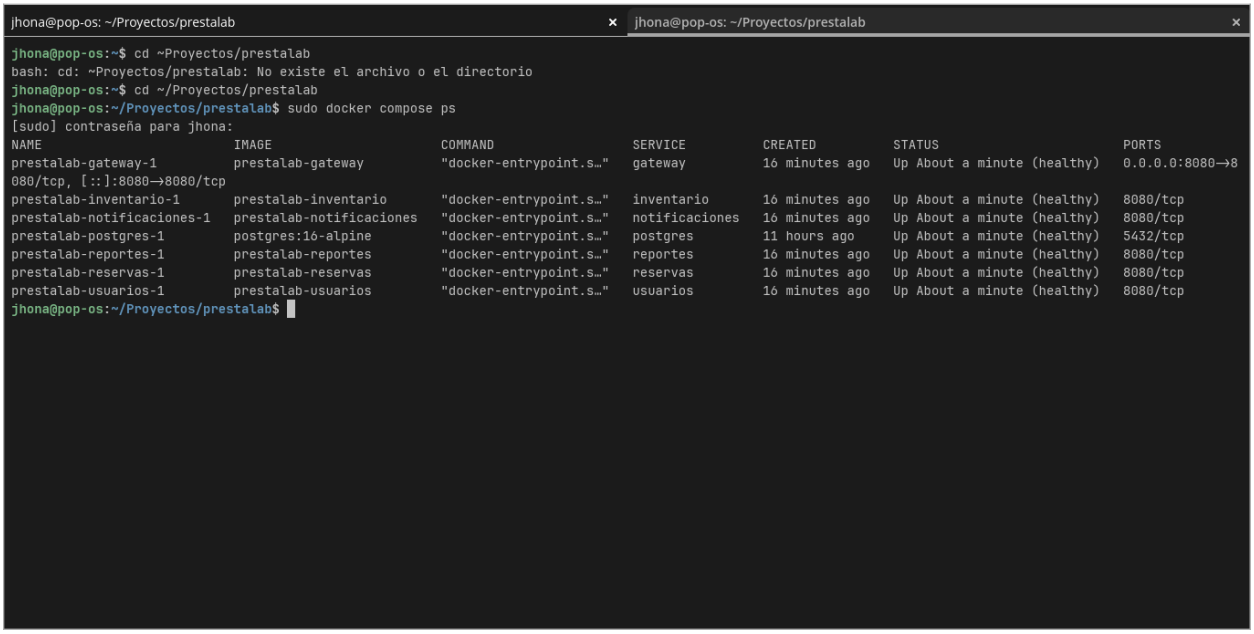
Capturas tomadas en el equipo de un integrante, con el sistema levantado en Docker, el 28 de octubre de 2026.



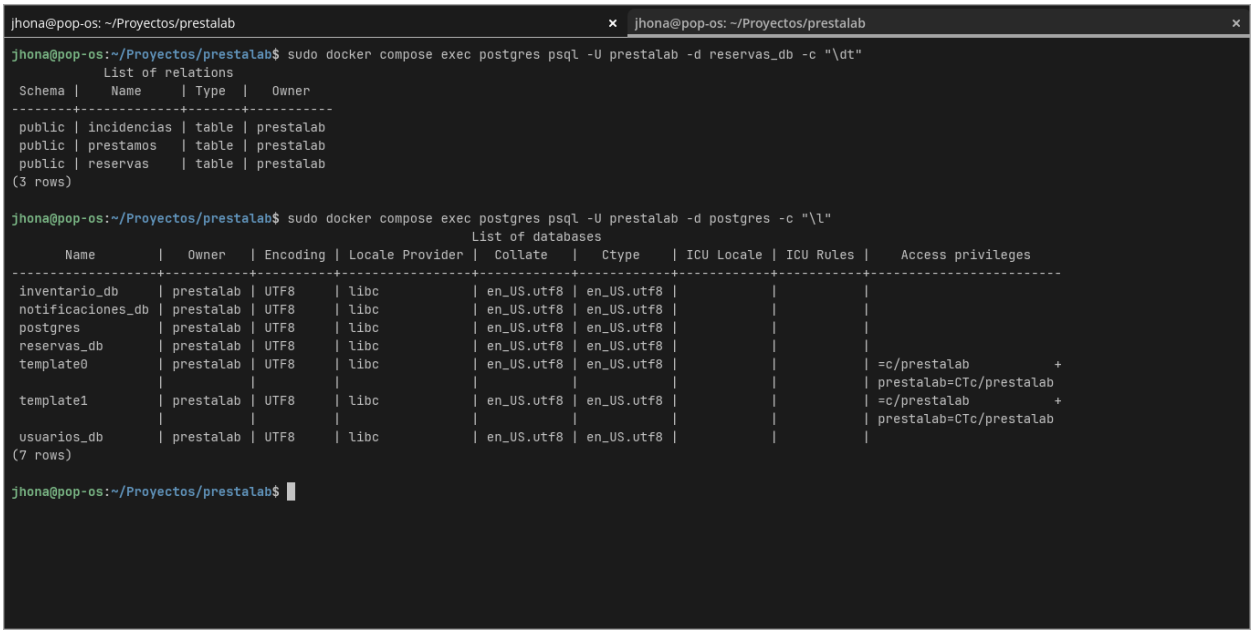

*Captura 1. Arranque con «docker compose up»: PostgreSQL inicia y los 5 servicios y el Gateway quedan escuchando en sus puertos (3001 a 3005 y 8080). El aviso «database "prestalab" does not exist» lo genera la revisión automática de Docker (pg_isready) y no es una falla.*



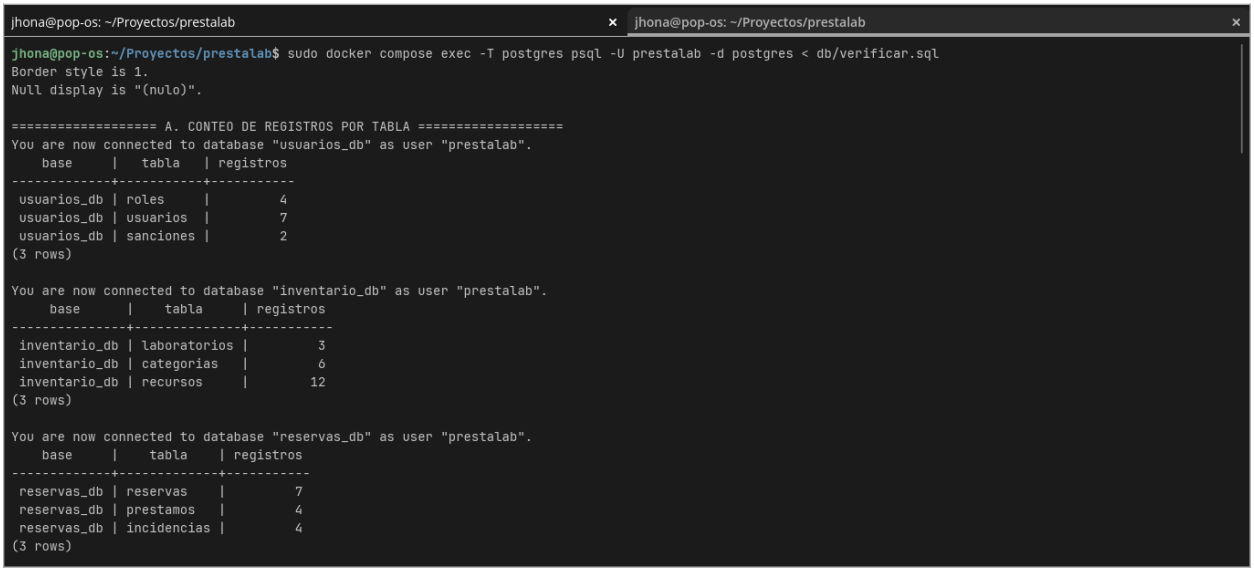

*Captura 2. http://localhost:8080/health: el Gateway y los 5 servicios responden «ok».*



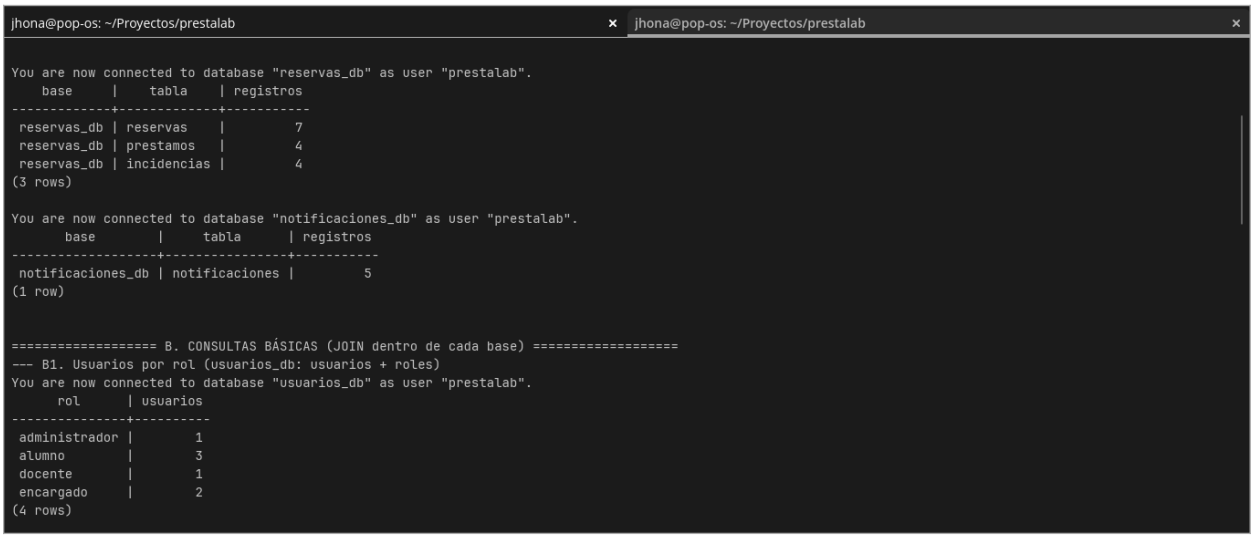

*Captura 3. «docker compose ps»: los 7 contenedores (PostgreSQL, Gateway y 5 servicios) están en ejecución y sanos (healthy).*



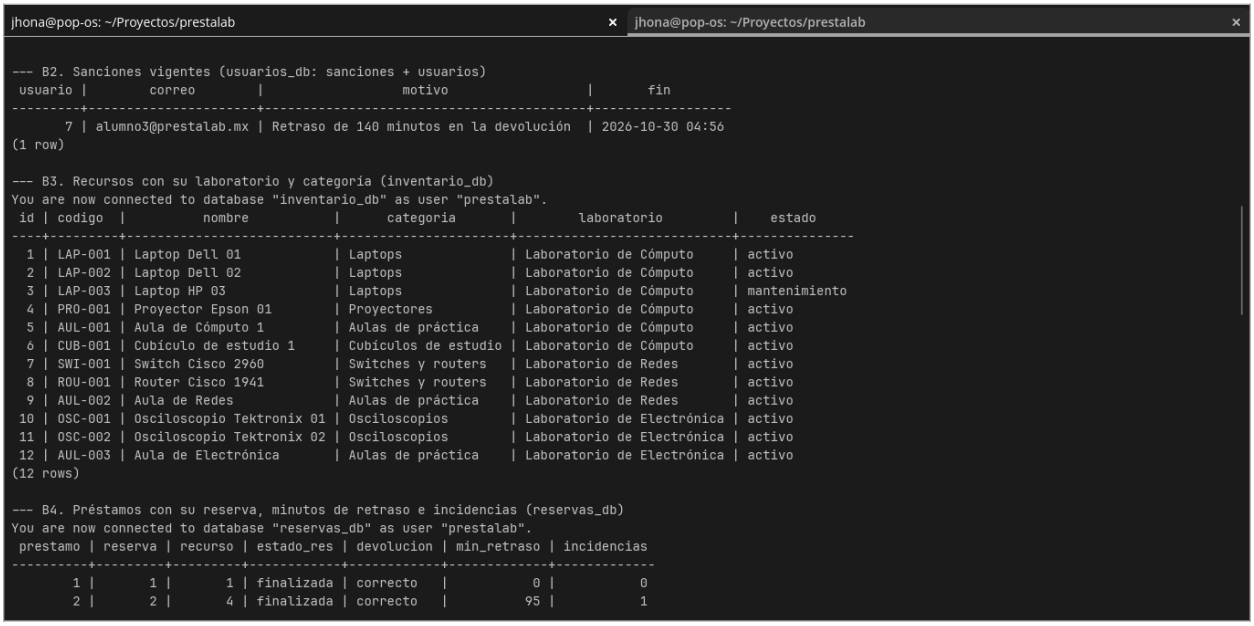

*Captura 4. Arriba, las tablas de reservas_db (reservas, prestamos e incidencias). Abajo, las 4 bases del proyecto: usuarios_db, inventario_db, reservas_db y notificaciones_db.*



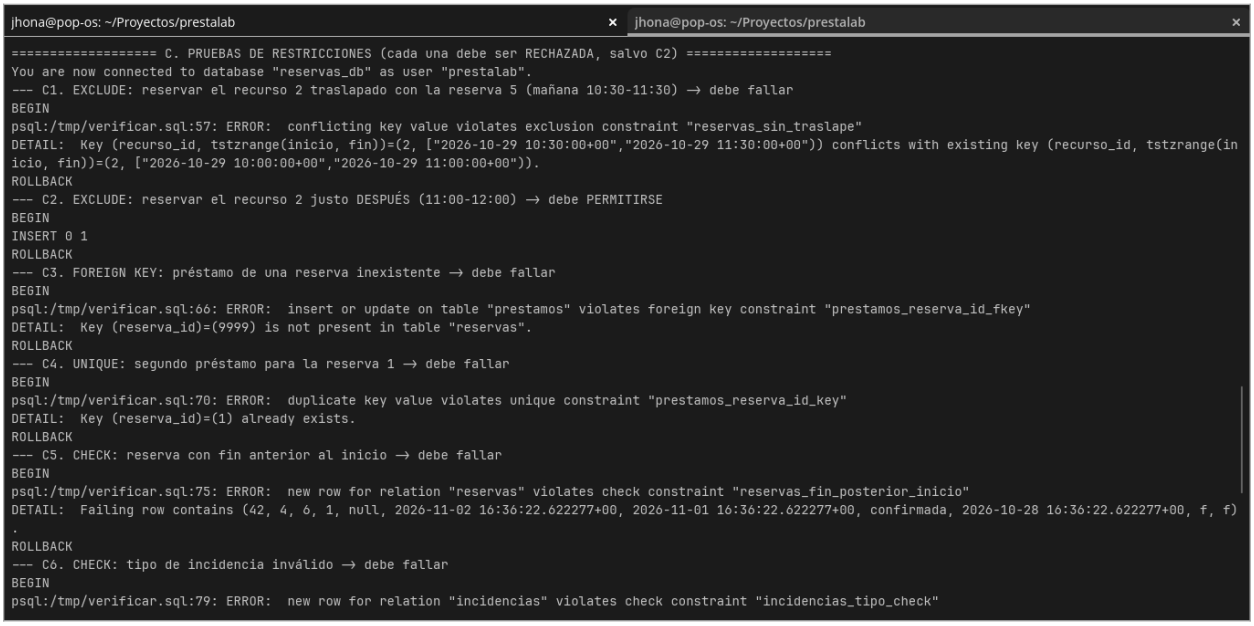

*Captura 5. verificar.sql, parte A: conteo de registros de usuarios_db (4 roles, 7 usuarios, 2 sanciones), inventario_db (3 laboratorios, 6 categorías, 12 recursos) y reservas_db (7 reservas, 4 préstamos, 4 incidencias).*



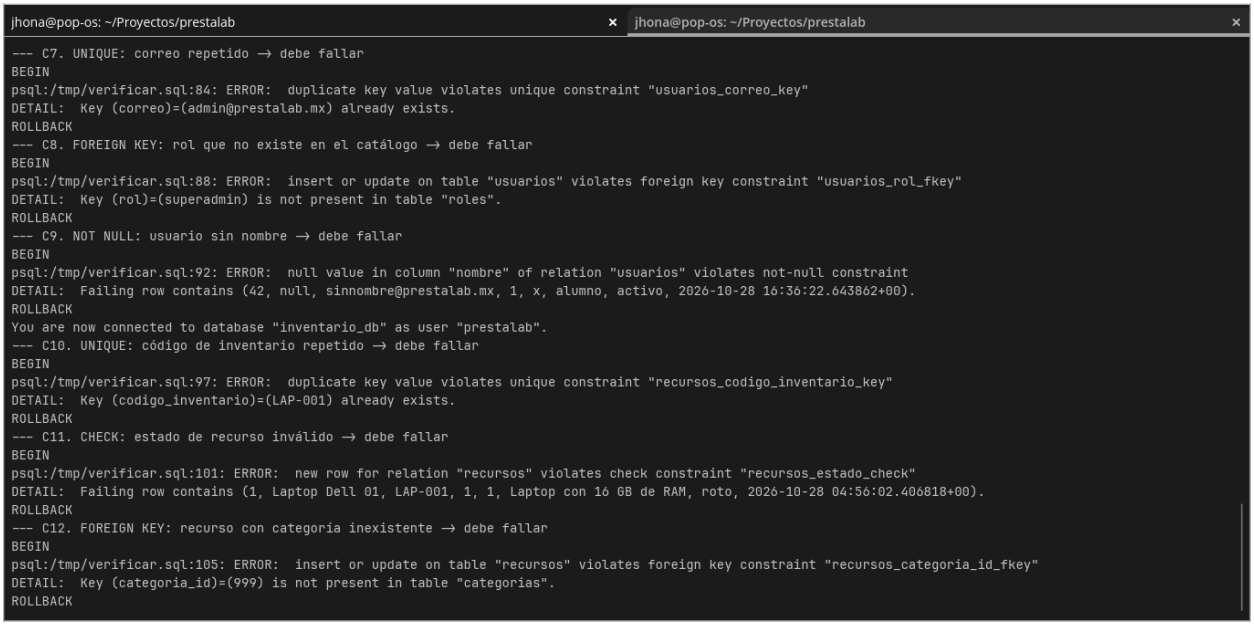

*Captura 6. Final de la parte A (notificaciones_db: 5 registros; 54 en total) e inicio de la parte B: consulta B1, usuarios por rol (une usuarios y roles).*




*Captura 7. Parte B: B2 muestra la sanción vigente del alumno3; B3 une recursos, categorías y laboratorios (12 recursos, la Laptop HP 03 en mantenimiento); B4 calcula los minutos de retraso de cada préstamo.*




*Captura 8. Parte C, pruebas C1 a C6: cada intento inválido es rechazado por PostgreSQL (ERROR) y se revierte (ROLLBACK). C2 sí se acepta (INSERT 0 1) porque el horario empieza justo cuando termina la otra reserva.*




*Captura 9. Parte C, pruebas C7 a C12: correo repetido, rol inexistente, usuario sin nombre, código de inventario repetido, estado inválido y categoría inexistente. Todas son rechazadas.*

## 9. Criterios de aceptación

| Criterio | Estado | Dónde se comprueba |
|---|---|---|
| **El modelo soporta las operaciones de la API** | Cumplido | Sección 6.1 |
| **No hay relaciones imposibles o ambiguas** | Cumplido | Sección 2.1 |
| **Los scripts se ejecutan desde cero** | Cumplido | Sección 7 |
| **El equipo entiende cada tabla y relación** | Por demostrar | Secciones 1 a 3 (en la defensa) |
| **Mínimo 5 tablas** | Cumplido | 10 tablas |
| **Mínimo 10 registros de prueba** | Cumplido | 54 registros |
| **Consultas básicas de verificación** | Cumplido | Sección 8 |
| **Restricciones UNIQUE, NOT NULL, CHECK o equivalentes** | Cumplido | Sección 4 |

## Referencias


- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: Constraints. https://www.postgresql.org/docs/16/ddl-constraints.html
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: Range Types. https://www.postgresql.org/docs/16/rangetypes.html
- Richardson, C. (2018). Microservices Patterns. Manning Publications.

- Newman, S. (2021). Building Microservices (2.ª ed.). O'Reilly Media.

## Anexo A. Bitácora de cambios

| Semana | Cambio realizado | Motivo | Responsable(s) | Evidencia |
|---|---|---|---|---|
| 1 | Versión inicial de la propuesta | Primera entrega | Equipo | Documento de la Entrega 1 |
| 2 | Estado administrativo separado de la disponibilidad | Evitar responsabilidades duplicadas entre Inventario y Reservas | Equipo | Documento de la Entrega 2 |
| 2 | 10 RF, 9 RNF, matriz de permisos, servicios y diagrama | Contenido de la Entrega 2 | Equipo | Documento de la Entrega 2 |
| 3 | 33 endpoints REST y rutas internas; formato único de errores | Contenido de la Entrega 3 | Equipo | Documento de la Entrega 3 y openapi.yaml |
| 3–4 | Recurso incluye tipo y encargadoId; rutas internas I-05 e I-06 de solo lectura; códigos 422 LABORATORIO_NO_VALIDO y CATEGORIA_NO_VALIDA | Necesidades encontradas al implementar; contrato y documento sincronizados | Equipo | openapi.yaml y Entrega 3 actualizados |
| 4 | Tabla roles con llave foránea desde usuarios | La entidad Rol de la Entrega 2 no tenía tabla propia | Equipo | Sección 5.1; evidencia C8 |
| 4 | Datos de prueba en archivos SQL con fechas relativas | Datos reproducibles y que no caducan | Equipo | Sección 9; archivos 002_datos_prueba.sql |
| 4 | Restricciones CHECK con nombre explícito | Mensajes de error claros y diccionario legible | Equipo | Sección 6 |
| 4 | Instalador SQL con rutas relativas (\ir) | Fallaba si se ejecutaba desde otra carpeta | Equipo | db/instalar.sql |
| 4 | Diagrama ER y diccionario generados desde el catálogo de PostgreSQL | Que la documentación no difiera de la base real | Equipo | scripts/extraer-modelo.js y generar-er.py |